Học thêm, bài tập và hướng dẫn giải

React — Học thêm về state, effect và ứng dụng quản lý khóa học

Ví dụ hướng đến React 18 đang dùng trong StudyOnline. Tài liệu bổ sung cho JSX, props, state, hooks, API và routing.

Mục tiêu học thêm

- Cập nhật state đúng khi phụ thuộc trạng thái trước
- Phân biệt dữ liệu suy ra, sự kiện và effect
- Xử lý request cũ và thiết kế trạng thái tải/lỗi/rỗng

1. State là ảnh chụp của lần render

Gọi setter yêu cầu lần render mới; biến state trong hàm xử lý hiện tại không lập tức biến thành giá trị mới. Khi cập nhật phụ thuộc giá trị trước, dùng updater function.

```
function Counter() {
  const [count, setCount] = useState(0);
  function addTwo() {
    setCount(c => c + 1);
    setCount(c => c + 1);
  }
  return <button onClick={addTwo}>{count}</button>;
}
```

Ví dụ cần import useState từ react. Hai updater được áp dụng liên tiếp nên tăng 2. Tránh thay đổi trực tiếp mảng/đối tượng trong state; tạo giá trị mới để React và các component con nhận ra thay đổi.

```
setCourses(previous => previous.map(course =>
  course.id === editedId ? { ...course, title: newTitle } : course
));
```

key khi render danh sách nên là mã ổn định. Dùng chỉ số khi thêm/xóa/sắp xếp có thể làm state của một hàng gắn nhầm sang hàng khác.

2. Khi nào cần effect?

Effect dùng đồng bộ với hệ thống bên ngoài như kết nối, timer hay request. Nếu chỉ lọc danh sách từ state hiện có, tính trong render thay vì lưu thêm state rồi cập nhật bằng effect.

```
const visibleCourses = courses.filter(course =>
  course.title.toLowerCase().includes(search.toLowerCase())
);
```

Thao tác lưu do người dùng bấm nút thường thuộc event handler. Effect không nên được dùng để suy đoán rằng người dùng vừa bấm nút chỉ vì state thay đổi. Với dependency, liệt kê các giá trị reactive thật sự được effect đọc; bỏ dependency tùy tiện dễ làm effect dùng dữ liệu cũ.

3. Request và cleanup

```
useEffect(() => {
  const controller = new AbortController();
  let active = true;
  setLoading(true);
  setError(null);
  fetch('/api/courses', { signal: controller.signal })
    .then(response => {
      if (!response.ok) throw new Error('Không tải được khóa học');
      return response.json();
    })
    .then(payload => { if (active) setCourses(payload.data || []); })
    .catch(error => {
      if (active && error.name !== 'AbortError') setError(error.message);
    })
    .finally(() => { if (active) setLoading(false); });
  return () => { active = false; controller.abort(); };
}, []);
```

Đây là phần đặt trong component có các state tương ứng; schema payload phải đối chiếu API thật. Cleanup ngăn request cũ cập nhật component không còn cần dữ liệu. Trong Strict Mode phát triển, effect có thể được chạy setup/cleanup thêm để phát hiện lỗi; cần cleanup đúng thay vì tắt Strict Mode để che lỗi.

4. Context, reducer và routing

Context chia sẻ dữ liệu như người dùng đăng nhập; không tự thay thế mọi state cục bộ. Reducer hữu ích khi nhiều thao tác phải cập nhật các phần liên quan theo quy tắc rõ ràng. Reducer nên thuần, không gọi API bên trong.

Frontend chặn route theo vai trò giúp trải nghiệm rõ ràng nhưng backend vẫn phải kiểm tra quyền mỗi request. Không xem việc ẩn nút sửa khóa là bảo vệ dữ liệu. Khi route đổi id khóa học, effect tải dữ liệu cần phụ thuộc id và hủy dữ liệu/request cũ phù hợp.

Bài tập và hướng dẫn

1. Dễ: giải thích hai lần setCount(count + 1) khác hai updater như thế nào. Khi cùng đọc ảnh chụp count, cả hai có thể yêu cầu cùng một giá trị tăng 1.
2. Trung bình: lọc danh sách theo từ khóa mà không thêm state visibleCourses. Tính bằng filter; giữ courses là danh sách gốc.
3. Khó: chuyển nhanh giữa hai trang chi tiết khóa khi mạng chậm. Chứng minh dữ liệu khóa cũ không ghi đè khóa mới; thử bằng làm chậm network và cleanup request.

Dự án mở rộng và tiêu chí

Làm trang quản lý khóa với xem/thêm/sửa/xóa, xác nhận thao tác xóa và trạng thái lỗi. Chỉ cập nhật thành công khi API thành công, hoặc triển khai cập nhật lạc quan kèm hoàn tác rõ ràng. Kiểm thử danh sách rỗng, lỗi 401, lỗi 403, gửi form hai lần và mất mạng. Component giao diện cần thể hiện tải/lỗi/rỗng/thành công; kiểm tra quyền cuối cùng thuộc backend.

Nguồn học tiếp

https://react.dev/learn/synchronizing-with-effects

https://react.dev/learn/queueing-a-series-of-state-updates

https://react.dev/learn/updating-arrays-in-state